TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐHQG HCM
KHOA ĐIỆN TỬ - VIỄN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC
Thực hành GIAO TIẾP MÁY TÍNH
Học kì 1 – Năm học: 2025 – 2026

Đề tài:
CẢM BIẾN NHIỆT ĐỘ VÀ ĐỘ ẨM

Sinh viên thực hiện : **LÊ HOÀNG BẢO NGỌC**
LÊ TRẦN ANH KIỆT
LÊ THANH VY
NGUYỄN VY NGỌC

Giảng viên : **HỒ THANH BẢO**

Khoa : **ĐIỆN TỬ VIỄN THÔNG**

Lớp : **22DTV_CLC1**

Tp. Hồ Chí Minh, tháng 12 năm 2025

PHIẾU ĐÁNH GIÁ ĐỒ ÁN
THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU

Phần bắt buộc

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	1		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		
Giải thích nguyên lý hoạt động	1		
Sử dụng ngắt, timer, uart, adc, …	0,5		
Sử dụng cơ sở dữ liệu	0,5		
Cuốn báo cáo trình bày đầy đủ nội dung	0.5		
Slide trình bày dễ hiểu	0.5		

Phần điểm cộng

Tiêu chí	Điểm	Tổng điểm tối đa	Đánh giá
Demo hoạt động tốt	0,5		
Giải thích nguyên lý và thuật toán	0,5		
Mức độ hoàn thiện và độ tin cậy của dữ liệu	1		

STT Nhóm: ….....
Điểm tổng kết: ….....

……., ngày … tháng … năm 2025
Người đại diện nhóm
(Ký tên, ghi rõ họ tên)

BẢNG PHÂN CÔNG

MSSV	Họ tên	Ngày giao	Công việc	Đánh giá
22207119	Lê Hoàng Bảo Ngọc	21/11/2025	Tìm hiểu thông số kỹ thuật của AHT20 để báo cáo	Hoàn thành đúng hạn
		29/11/2025	Hỗ trợ chạy đồ án (hỗ trợ đọc datasheet linh kiện để nối chân PIN, test phần quảng bá và giao tiếp PC)	Hoàn thành đúng hạn
		02/12/2025	Chuẩn bị slide powerpoint thuyết trình (cập nhật các thông tin từ file báo cáo)	Hoàn thành đúng hạn
		13+14/12/2025	Cập nhật powerpoint hoàn thiện	Hoàn thành đúng hạn
22207116	Lê Trần Anh Kiệt	21/11/2025	Nghiên cứu datasheet để nối chân PIN, viết file driver AHT20, nghiên cứu phần giao tiếp I2C	Hoàn thành đúng hạn
		30/11/2025	Viết chương trình cho hệ thống (thực hiện được các tính năng cơ bản trước như hiển thị nhiệt độ và độ ẩm trên LCD và quảng bá BLE, đọc giá trị và xuất ra file CSV)	Hoàn thành đúng hạn
		02/12/2025	Cập nhật hiển thị kết quả giao tiếp với chu kỳ đọc sensor và thời gian quảng bá trên LCD, thêm các command trong phần giao tiếp UART	Hoàn thành đúng hạn
		06/12/2025	Bổ sung cho đồ án phần SQLite	Hoàn thành đúng hạn
22207107	Lê Thanh Vy	29/11/2025	Lo về phần cứng (mua sensor và hàn chân), hỗ trợ đọc datasheet linh kiện để nối chân PIN và chạy đồ án	Hoàn thành đúng hạn
		08/12/2025	Kiểm tra thuật toán hệ thống	Hoàn thành đúng hạn
		13+14/12/2025	Hỗ trợ hoàn thiện phần powerpoint, chuẩn bị phần thuyết trình	Hoàn thành đúng hạn
22207064	Nguyễn Vy Ngọc	21/11/2025	Tìm hiểu thông số kỹ thuật của AHT20 để báo cáo	Hoàn thành đúng hạn
		02/12/2025	Chuẩn bị bài báo cáo (soạn phần tổng quan và kết luận)	Hoàn thành đúng hạn
		05/12/2025	Tiếp tục tìm thông tin về các linh kiện như LCD, nghiên cứu và giải thích thuật toán của đồ án để viết báo cáo	Hoàn thành đúng hạn
		08/12/2025	Cập nhật phần giải thích thuật toán hiển thị trên LCD, quảng bá BLE, giao tiếp với PC (xuất ra file CSV)	Hoàn thành đúng hạn
		12/12/2025	Cập nhật phần giao tiếp với PC thông qua giao tiếp UART (bổ sung SQLite)	Hoàn thành đúng hạn

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG 1
1.1. MÔ TẢ CHUNG 1
1.2. THÔNG SỐ KỸ THUẬT 1
1.2.1. Phần cứng (Hardware) 1
1.2.2. Phần mềm (Software) 2
1.3. CHỨC NĂNG CỦA HỆ THỐNG 2
1.3.1. Luồng dữ liệu 2
1.3.2. Chức năng chi tiết 2
CHƯƠNG 2: CẢM BIẾN AHT20 4
2.1. GIỚI THIỆU CHUNG 4
2.2. CẤU TRÚC FILE DRIVER 5
2.3. KHỞI TẠO 5
2.4. ĐỌC DỮ LIỆU 6
CHƯƠNG 3: HIỂN THỊ LÊN LCD 8
3.1. TỔNG QUAN 8
3.2. TRIỂN KHAI CHƯƠNG TRÌNH 8
CHƯƠNG 4: TRUYỀN DỮ LIỆU ĐẾN PC 11
4.1. TỔNG QUAN 11
4.2. CẤU TRÚC FILE `aht20_logger.c` 11
4.2.1. Cấu hình cổng COM 11
4.2.2. Gửi và nhận dữ liệu UART 12
4.2.3. Phân tích dữ liệu cảm biến 13
4.2.4. Lưu vào cơ sở dữ liệu 13
4.3. CƠ SỞ DỮ LIỆU SQLite 14
4.3.1. Cấu trúc và khởi tạo 14
4.3.2. Lưu trữ dữ liệu vào SQLite 15
CHƯƠNG 5: QUẢNG BÁ BLE (BLE ADVERTISING) 17
5.1. GIỚI THIỆU 17
5.2. CẤU TRÚC CUSTOM ADVERTISING DATA 17
5.3. CẬP NHẬT CẢM BIẾN 18
5.4. XỬ LÝ SỰ KIỆN BLE (BLE EVENT HANDLER) 19

5.5. HÀM START ADVERTISING 20
CHƯƠNG 6: CHƯƠNG TRÌNH CHÍNH 22
6.1. TỔNG QUAN 22
6.2. KIẾN TRÚC PHẦN MỀM 22
6.2.1. UART 22
6.2.2. Timer Callback 23
6.2.3. Vòng lặp chính 24
6.3. CHẾ ĐỘ HOẠT ĐỘNG 25
CHƯƠNG 7: CHẠY THỬ NGHIỆM 26
7.1. BẢNG ĐIỀU KHIỂN VÀ GIAO DIỆN CONSOLE 26
7.2. CHẠY CÁC LỆNH COMMAND 27
7.3. KẾT QUẢ 32
CHƯƠNG 8: KẾT LUẬN 34
8.1. TỔNG KẾT 34
8.2. HẠN CHẾ 34
8.3. HƯỚNG PHÁT TRIỂN 34

DANH MỤC HÌNH

Hình 1.1. EFR32xG21 2.4 GHz 10 dBm và Wireless Starter Kit Mainboard 1
Hình 2.1. Cảm biến AHT20 4
Hình 2.2. Thông số kỹ thuật cơ bản của Cảm biến AHT20 4
Hình 7.1. Bảng điều khiển 26
Hình 7.2. Kết quả hiển thị trên giao diện Console 27
Hình 7.3. Kết quả khi chạy Command ‘1’ 27
Hình 7.4. Kết quả khi chạy Command ‘2’ 27
Hình 7.5. Kết quả khi chạy Command ‘3’ 28
Hình 7.6. Kết quả khi chạy Command ‘4’ và ‘5’ 28
Hình 7.7. Giảm thời gian quảng bá BLE 100ms 29
Hình 7.8. Tăng thời gian quảng bá BLE 100ms 29
Hình 7.9. Kết quả quảng bá trên BLE 30
Hình 7.10. Kết quả khi chạy Command ‘6’ 30
Hình 7.11. Kết quả khi chạy Command ‘7’ 31
Hình 7.12. Kết quả chạy Command ‘q’ 31
Hình 7.13. Dữ liệu được in ra trong file CSV 32
Hình 7.14. Cấu trúc file `sensor_data.db` 32
Hình 7.15. Ghi dữ liệu trên SQLite 33
Hình 7.16. Kết quả hiển thị lên LCD 33

DANH MỤC BẢNG

Bảng 3.1. Thông số kỹ thuật cơ bản màn hình LCD 8
Bảng 3.2. Thiết kế mô hình 4 lớp 8
Bảng 4.1. Cấu trúc cơ sở dữ liệu SQLite 14
Bảng 5.1. Cấu trúc cơ bản của một gói tin BLE Advertising 17

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG

1.1. MÔ TẢ CHUNG

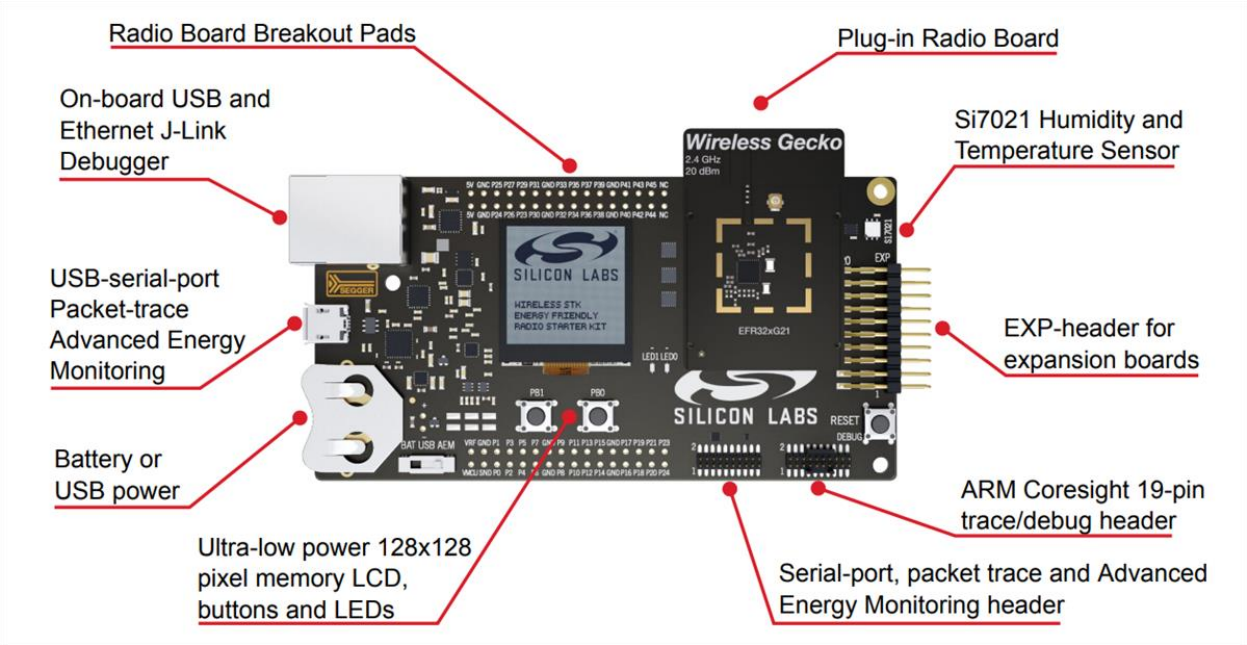
Hệ thống các thiết bị được sử dụng nhằm mục đích theo dõi những biến đổi của môi trường xung quanh đã không còn quá xa lại đối với người tiêu dùng trong xã hội hiện đại. Và với sự sáng tạo không ngừng cùng công nghệ ngày càng hiện đại, những hệ thống này sẽ không chỉ dừng lại ở "theo dõi sự biến đổi".

Vì lẽ đó, nhóm chúng em mong muốn xây dựng một hệ thống không chỉ theo dõi, hiển thị dữ liệu trên màn hình LCD chỉ số nhiệt độ và độ ẩm của môi trường mà còn tích hợp thêm chức năng gửi dữ liệu đến máy tính cá nhân (Personal Computer - PC) thông qua truyền thông UART và ghi vào hai cơ sở dữ liệu CSV và SQLite. Ngoài ra, dữ liệu có thể được quảng bá đến thiết bị di động qua BLE (Bluetooth Low Energy).

1.2. THÔNG SỐ KỸ THUẬT

1.2.1. Phần cứng (Hardware)

Trong phạm vi đồ án môn học này, chúng em sử dụng bo mạch EFR32xG21 2.4 GHz 10 dBm (BRD4180B Radio Board) cùng với Wireless Starter Kit Mainboard (BRD4001A) của Silicon Labs và cảm biến đo nhiệt độ - độ ẩm AHT20.



Hình 1.1. EFR32xG21 2.4 GHz 10 dBm và Wireless Starter Kit Mainboard

Cảm biến AHT20 có độ chính xác cao giao tiếp theo chuẩn I2C cho phép đo nhiệt độ và độ ẩm với sai số lần lượt là ±0.3°C và ±2% RH. Trong đồ án này, giá trị nhiệt độ - độ ẩm đo được từ cảm biến sẽ được đọc bởi vi điều khiển thông qua I2C và truyền đến PC bằng giao tiếp UART 115200 baud. Cảm biến có khích thước gọn nhẹ, độ ổn định cao với sai số nhỏ phù hợp với ý tưởng xây dựng đồ án môn học của nhóm chúng em.

1.2.2. Phần mềm (Software)

Nhóm chúng em sử dụng công cụ Simplicity Studio 5 để build chương trình sử dụng SDK "Simplicity SDK Suite v2024.6.2". Bên cạnh đó nhóm chúng em sử dụng phần mềm Visual Studio Code để giao tiếp với PC thông qua UART và lưu trữ vào hai cơ sở dữ liệu là CSV và SQLite. Về việc quảng bá BLE thì sử dụng app Si Connect trên điện thoại để xem quá trình cập nhật kết quả.

1.3. CHỨC NĂNG CỦA HỆ THỐNG

1.3.1. Luồng dữ liệu

Chỉ số về nhiệt độ và độ ẩm của môi trường sẽ được đọc từ cảm biến AHT20 mỗi 5s và truyền dữ liệu đến EFR32xG21 thông qua giao thức I2C. Tại EFR32xG21, dữ liệu thô sẽ được xử lý (làm tròn và lưu trữ) và hiển thị lên màn hình Console.

Ở bước kế tiếp, dữ liệu về nhiệt độ - độ ẩm sẽ được hiển thị lên màn hình LCD và gửi qua UART đến PC. Ngoài ra, dữ liệu cũng sẽ được quảng bá đến thiết bị di động qua BLE sử dụng ứng dụng Si Connect. Cuối cùng, các chỉ số sẽ được lưu vào cơ sở dữ liệu CSV và SQLite.

1.3.2. Chức năng chi tiết

- *Chế độ hoạt động*

1. Auto-Logging Mode: hệ thống sẽ tự động đọc và gửi dữ liệu.

2. Command Mode: hệ thống sẽ nhận lệnh thông qua UART được gửi từ PC.

3. BLE Broadcast: dữ liệu sẽ được quảng bá liên tục qua bluetooth mà không cần kết nối phần cứng.

- *Các lệnh điều khiển:*

'1' → Đọc dữ liệu từ cảm biến ngay lập tức.

'2' → Chu kì đọc dữ liệu từ cảm biến tăng 1s.

'3' → Chu kì đọc dữ liệu từ cảm biến giảm 1s.

'4' → Chu kì quảng bá BLE tăng 100ms.

'5' → Chu kì quảng bá BLE giảm 100ms.

'6' → Hiển thị cấu hình hiện tại.

'7' → Hiển thị cơ sở dữ liệu.

'h' → Hiển thị menu điều khiển.

'q' → Thoát chương trình.

CHƯƠNG 2: CẢM BIẾN AHT20

2.1. GIỚI THIỆU CHUNG

Cảm biến AHT20 là cảm biến số với chức năng đo lường nhiệt độ và độ ẩm của môi trường, giao tiếp với vi điều khiển thông qua chuẩn I2C tiêu thụ năng lượng thấp. Cảm biến được tích hợp bộ ADC và mạch bù lỗi giúp đảm bảo cung cấp dữ liệu có độ chính xác cao và ổn định trong nhiều điều kiện khác nhau. Do đó, cảm biến AHT20 là sự lựa chọn phù hợp cho các ứng dụng đo đạc chỉ số môi trường, tự động hóa, thiết bị IoT và hệ thống giám sát thời tiết.

Hình 2.1. Cảm biến AHT20

Sensor Performance

Relative Humidity

Parameter	Condition	Min	Typical	Max	Unit
resolution ratio	Typical		0.024		%RH
accuracy error[1]	Typical		±2		%RH
	Max	Figure 2			%RH
Repeatability			±0.1		%RH
Hysteresis			±1		%RH
Nonlinear			<0.1		%RH
Response time[2]	t 63%		8		S
Scope of work	extended[3]	0		100	%RH
Long time drift[4]	Normal		<0.5		%RH/yr

Table 1 Humidity Characteristic

Temperature

Parameter	Condition	Min	Typical	Max	Unit
resolution ratio	Typical		0.01		°C
accuracy error[1]	Typical		±0.3		°C
	Max	Figure 3			°C
Repeatability			±0.1		°C
Hysteresis			±0.1		°C
Response time[6]	t 63%	5		30	S
Scope of work	extended[3]	-40		85	°C
Long time drift			<0.04		°C/yr

Table 3 Temperature Characteristic

Hình 2.2. Thông số kỹ thuật cơ bản của Cảm biến AHT20

2.2. CẤU TRÚC FILE DRIVER

Driver của cảm biến AHT20 đóng vai trò trung gian giữa lớp phần cứng và lớp ứng dụng (application layer) nhằm cung cấp các hàm API giúp người lập trình có thể đọc nhiệt độ – độ ẩm mà không cần xử lý trực tiếp các thao tác mức thấp của giao tiếp I2C.

Driver sẽ chịu trách nhiệm định nghĩa các hằng số cần thiết, các câu lệnh điều khiển và bit trạng thái để thuận tiện trong quá trình thực hiện các thao tác đọc – ghi thanh ghi cảm biến cũng như giám sát quá trình hiệu chuẩn và sẵn sàng của cảm biến. Quá trình xử lý dữ liệu thô và chuyển đổi sang giá trị vật lý như nhiệt độ (°C) và độ ẩm (%RH) cũng được thực hiện ở driver. Nhờ đó, ở lớp ứng dụng chỉ cần gọi hàm thực hiện chức năng mà không cần quan tâm đến các gói dữ liệu, bit lệnh hay cấu trúc frame I2C.

File driver thường bao gồm hai thành phần chính:

- File Header `aht20.h` chứa khai báo hằng số, câu lệnh điều khiển

```
// AHT20 I2C Address
#define AHT20_I2C_ADDRESS       0x38

// AHT20 Commands
#define AHT20_CMD_INITIALIZE    0xBE
#define AHT20_CMD_TRIGGER       0xAC
#define AHT20_CMD_SOFT_RESET    0xBA

// AHT20 Status bits
#define AHT20_STATUS_BUSY       0x80
#define AHT20_STATUS_CALIBRATED 0x08
```

Khai báo các hàm API chính:

```
sl_status_t aht20_init(sl_i2cspm_t *i2cspm);
sl_status_t aht20_read_data(sl_i2cspm_t *i2cspm, float *temperature, float
*humidity);
sl_status_t aht20_soft_reset(sl_i2cspm_t *i2cspm);
```

- File thực thi `aht20.c` có nhiệm vụ thực hiện các hàm API đã được khai báo trong file header. Câu lệnh điều khiển kiểm tra liên tục trạng thái của cảm biến để bắt được sự thay đổi trong môi trường, xử lý những tín hiệu thô 20-bit nhận được từ cảm biến và chuyển đổi giá trị ADC sang các đơn vị vật lý thông dụng (quá trình gửi và nhận dữ liệu đều thông qua I2C).

2.3. KHỞI TẠO

Hàm `aht20_init()` được thiết kế với mục đích đưa cảm biến về trạng thái hoạt động ổn định ngay sau khi cấp nguồn, do cảm biến AHT20 chưa sẵn sàng đo ngay thời điểm khởi động mà cần thực hiện hiệu chuẩn nội bộ. Hàm khởi tạo sẽ kiểm tra sự tồn tại của cảm biến trên bus I2C, chờ ổn định nguồn khoảng 40ms để thực hiện gửi các lệnh hiệu chuẩn.

Hàm khởi tạo sẽ gửi lệnh INIT nhằm thực hiện kiểm tra sự tồn tại của cảm biến trên I2C, gửi lệnh yêu cầu cảm biến thực hiện quá trình hiệu chuẩn và chờ đến khi cảm biến sẵn sàng (bit BUSY = 0, bit CALIBRATED = 1). Sau khi tất cả quá trình đã hoàn thành, cảm biến sẽ chuyển sang chế độ sẵn sàng chờ lệnh đo.

2.4. ĐỌC DỮ LIỆU

Hệ thống gọi hàm `aht20_read()` để đọc chỉ số nhiệt độ và độ ẩm

```
sl_status_t aht20_read_data(sl_i2cspm_t *i2cspm, float *temperature, float
*humidity)
{
  I2C_TransferSeq_TypeDef seq;
  I2C_TransferReturn_TypeDef ret;
  uint8_t cmd[3] = {AHT20_CMD_TRIGGER, 0x33, 0x00};
  uint8_t data[7];
  uint32_t raw_humidity;
  uint32_t raw_temperature;

  // Trigger measurement
  seq.addr = AHT20_I2C_ADDRESS << 1;
  seq.flags = I2C_FLAG_WRITE;
  seq.buf[0].data = cmd;
  seq.buf[0].len = 3;

  ret = I2CSPM_Transfer(i2cspm, &seq);

  if (ret != i2cTransferDone) {
    return SL_STATUS_TRANSMIT;
  }

  // Wait for measurement (80ms typical)
  sl_sleeptimer_delay_millisecond(80);

  // Read measurement data
  seq.flags = I2C_FLAG_READ;
  seq.buf[0].data = data;
  seq.buf[0].len = 7;

  ret = I2CSPM_Transfer(i2cspm, &seq);

  if (ret != i2cTransferDone) {
    return SL_STATUS_RECEIVE;
  }

  // Check if busy
  if (data[0] & AHT20_STATUS_BUSY) {
    return SL_STATUS_BUSY;
  }

  // Calculate humidity: 20-bit value from bytes 1,2,3 (high 4 bits)
```

```
  raw_humidity = ((uint32_t)data[1] << 12) |
                 ((uint32_t)data[2] << 4) |
                 ((uint32_t)data[3] >> 4);

  // Calculate temperature: 20-bit value from bytes 3 (low 4 bits), 4, 5
  raw_temperature = (((uint32_t)data[3] & 0x0F) << 16) |
                    ((uint32_t)data[4] << 8) |
                    ((uint32_t)data[5]);

  // Convert to actual values
  *humidity = ((float)raw_humidity / 1048576.0) * 100.0;
  *temperature = (((float)raw_temperature / 1048576.0) * 200.0) - 50.0;

  return SL_STATUS_OK;
}
```

1. Gửi lệnh đo

Lệnh đo sẽ được gửi kèm cấu hình chi tiết như chọn chế độ đo đồng thời nhiệt độ - độ ẩm cũng như thiết lập tốc độ nhận dữ liệu theo chu kỳ.

2. Chờ cảm biến xử lý

Cảm biến AHT20 sẽ thực hiện lấy mẫu dữ liệu thô từ môi trường và bit BUSY được sử dụng với mục đích theo dõi trạng thái của cảm biến.

3. Đọc dữ liệu trả về

Byte	**Nội dung**
0	Trạng thái của cảm biến
1 – 3	Chỉ số độ ẩm
4 – 5	Chỉ số nhiệt độ

4. Xử lý dữ liệu

Độ ẩm (0 – 100%):

$$humidity = \frac{raw_humidity}{1048576.0} \times 100.0$$

Nhiệt độ (-40°C đến +125°C):

$$temperature = \left(\frac{raw_temperature}{1048576.0} \times 200.0\right) - 50.0$$

CHƯƠNG 3: HIỂN THỊ LÊN LCD

3.1. TỔNG QUAN

Bảng 3.1. Thông số kỹ thuật cơ bản màn hình LCD

Độ phân giải	128×128 pixels
Loại hiển thị	Monochrome (1-bit/pixel)
Giao tiếp	SPI
Tiêu thụ điện	< 15µW (ultra-low power)

Bảng 3.2. Thiết kế mô hình 4 lớp

Application (`app_lcd.c`)	Xử lý logic, quản lý hiển thị dữ liệu cảm biến
Graphic Library (GLIB)	Cung cấp hàm cần thiết cho hiển thị lên LCD (font, căn chỉnh,...)
Display Driver (DMD)	Khởi tạo LCD và quản lý màn hình
SPI Communication (`lcd.c`)	Giao thức truyền dữ liệu

3.2. TRIỂN KHAI CHƯƠNG TRÌNH

- *Khởi tạo hệ thống*

```
void memlcd_app_init(void)
{
  uint32_t status;

  // Enable the memory LCD
  status = sl_board_enable_display();
  EFM_ASSERT(status == SL_STATUS_OK);

  // Initialize the DMD support for memory LCD display
  status = DMD_init(0);
  EFM_ASSERT(status == DMD_OK);

  // Initialize the GLIB context
  status = GLIB_contextInit(&glibContext);
  EFM_ASSERT(status == GLIB_OK);

  glibContext.backgroundColor = White;
  glibContext.foregroundColor = Black;

  // Fill LCD with background color
  GLIB_clear(&glibContext);

  // Use Narrow font
  GLIB_setFont(&glibContext, (GLIB_Font_t *)&GLIB_FontNarrow6x8);
```

```
  // Initial display
  display_sensor_data();
}
```

- *Hiển thị dữ liệu*

```
static void display_sensor_data(void)
{
  char str[30];

  GLIB_clear(&glibContext);

  currentLine = 0;

  // === HEADER ===
  GLIB_drawStringOnLine(&glibContext, "AHT20 SYSTEM",
                        currentLine++, GLIB_ALIGN_CENTER, 0, 0, true);
  currentLine++;

  // === SENSOR DATA ===
  snprintf(str, sizeof(str), "Temp: %d C", g_temperature); //nhiệt độ
  GLIB_drawStringOnLine(&glibContext, str, currentLine++,
                        GLIB_ALIGN_LEFT, 5, 0, true);

  snprintf(str, sizeof(str), "Humi: %d %%", g_humidity); //độ ẩm
  GLIB_drawStringOnLine(&glibContext, str, currentLine++,
                        GLIB_ALIGN_LEFT, 5, 0, true);

  currentLine++;  // Khoảng trống

  // === PERIODS ===
  snprintf(str, sizeof(str), "Sensor: %d sec", g_sensor_period);
  GLIB_drawStringOnLine(&glibContext, str, currentLine++,
                        GLIB_ALIGN_LEFT, 5, 0, true);

  // HIỂN THỊ BLE INTERVAL - CHỈ DÙNG INT (ms)
  int ble_interval_ms = (g_ble_adv_interval * 625) / 1000;
  snprintf(str, sizeof(str), "BLE: %d ms", ble_interval_ms);
  GLIB_drawStringOnLine(&glibContext, str, currentLine++,
                        GLIB_ALIGN_LEFT, 5, 0, true);

  DMD_updateDisplay(); // cập nhật màn hình
}
```

- *Vòng lặp cập nhật*

```
void memlcd_app_process_action(void)
{
  static int last_temp = -999;
  static int last_humi = -999;
```

```
  static uint16_t last_sensor_period = 0;
  static uint16_t last_ble_interval = 0;

  // Cập nhật màn hình LCD khi BẤT KỲ giá trị nào thay đổi
  if (g_temperature != last_temp ||
      g_humidity != last_humi ||
      g_sensor_period != last_sensor_period ||
      g_ble_adv_interval != last_ble_interval) {

    last_temp = g_temperature;
    last_humi = g_humidity;
    last_sensor_period = g_sensor_period;
    last_ble_interval = g_ble_adv_interval;

    display_sensor_data();
  }
}
```

Biến `static` được sử dụng với mục đích lưu giá trị cũ và chỉ cập nhật chỉ số trên màn hình khi dữ liệu thay đổi giúp tối ưu hóa năng lượng tiêu thụ.

CHƯƠNG 4: TRUYỀN DỮ LIỆU ĐẾN PC

4.1. TỔNG QUAN

Chương trình được lập trình dựa trên ngôn ngữ C chạy trên nền tảng Window thực hiện chức năng kết nối với EFR32xG21 qua cổng COM (giao thức UART) nhằm tự động thu thập dữ liệu từ cảm biến và lưu vào cơ sở dữ liệu trên PC (CSV và SQLite). Ngoài ra, PC còn đóng vai trò như một bảng điều khiển hiển thị đầy đủ menu tương tác và gửi lệnh đến EFR32xG21.

4.2. CẤU TRÚC FILE `aht20_logger.c`

4.2.1. Cấu hình cổng COM

```
HANDLE open_port(const char* device, unsigned long baud_rate) {
    HANDLE port = CreateFileA(device, GENERIC_READ | GENERIC_WRITE, 0,
NULL,
        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);

    if (port == INVALID_HANDLE_VALUE) {
        printf("[ERROR] Cannot open %s\n", device);
        return INVALID_HANDLE_VALUE;
    }

    FlushFileBuffers(port);

    // TIMEOUT configuration - tăng để đọc response command
    COMMTIMEOUTS timeouts = { 0 };
    timeouts.ReadIntervalTimeout = 50;           // Tăng lên 50ms
    timeouts.ReadTotalTimeoutConstant = 200;     // Tăng lên 200ms
    timeouts.ReadTotalTimeoutMultiplier = 10;
    timeouts.WriteTotalTimeoutConstant = 100;
    timeouts.WriteTotalTimeoutMultiplier = 0;

    if (!SetCommTimeouts(port, &timeouts)) {
        printf("[ERROR] Failed to set timeouts\n");
        CloseHandle(port);
        return INVALID_HANDLE_VALUE;
    }

    // Configure serial port
    DCB dcb = { 0 };
    dcb.DCBlength = sizeof(DCB);
    dcb.BaudRate = baud_rate;
    dcb.ByteSize = 8;
    dcb.Parity = NOPARITY;
    dcb.StopBits = ONESTOPBIT;

    if (!SetCommState(port, &dcb)) {
```

```
        printf("[ERROR] Failed to configure port\n");
        CloseHandle(port);
        return INVALID_HANDLE_VALUE;
    }

    printf("[OK] Port opened: %s at %d baud\n", device, baud_rate);
    return port;
}
```

Hàm `open_port()` thiết lập kết nối với vi điều khiển sử dụng Windows API `CreateFileA()`, lưu trữ tên thiết bị (cổng COM kết nối) và tốc độ baud.

4.2.2. Gửi và nhận dữ liệu UART

Sử dụng hàm `uart_transmit()` để gửi lệnh từ PC đến vi điều khiển thông báo truyền dữ liệu.

```
int uart_transmit(HANDLE port, unsigned char* TX_buf, unsigned int
TX_buf_len) {
    DWORD writtenbytes = 0;
    // Gửi data
    if (!WriteFile(port, TX_buf, TX_buf_len, &writtenbytes, NULL)) {
        printf("[ERROR] WriteFile failed\n");
        return -1;
    }
    // Kiểm tra data (bytes) đã gửi
    if (writtenbytes == 0) {
        printf("[ERROR] WriteFile timeout\n");
        return -1;
    }

    // Đảm bảo data được gửi đi
    FlushFileBuffers(port);

    return 0;
}
```

Hàm `uart_receive_line()` sẽ đọc từng dòng dữ liệu (đánh dấu kết thúc dòng bằng `\n`) và lưu trạng thái của cảm biến giữa các lần gọi vào biến `static`. Thuật toán sẽ kiểm tra `\n` và thêm ký tự kết thúc `\0`, thời gian Timeout sẽ được kiểm tra bằng `GetTickCount()`.

```
int uart_receive_line(HANDLE port, char* buffer, int max_len, int
timeout_ms) {
    DWORD nbbytes = 0;
    static int total_bytes = 0;
    static char temp_buffer[BUFFER_SIZE];
    DWORD start_time = GetTickCount();

    while (total_bytes < max_len - 1) {
        if (GetTickCount() - start_time > timeout_ms) {
```

```
            return 0;  // Timeout
        }

        if (ReadFile(port, &temp_buffer[total_bytes], 1, &nbbytes, NULL))
{
            if (nbbytes > 0) {
                if (temp_buffer[total_bytes] == '\n') {
                    temp_buffer[total_bytes + 1] = '\0';
                    strcpy(buffer, temp_buffer);
                    int len = total_bytes + 1;
                    total_bytes = 0;
                    return len;
                }
                total_bytes++;
            }
        }
    }

    return 0;
}
```

4.2.3. Phân tích dữ liệu cảm biến

```
void process_sensor_data(const char* line) {
    int temperature = 0, humidity = 0;

    // Parse các format có thể
    if (sscanf(line, "[INFO] Temp: %d C, Humidity: %d %%",
              &temperature, &humidity) == 2) {
        save_to_database(temperature, humidity);
        return;
    }

    if (sscanf(line, "Temperature: %d C, Humidity: %d %%",
              &temperature, &humidity) == 2) {
        save_to_database(temperature, humidity);
        return;
    }
}
```

Hàm `process_sensor_data()` có nhiệm vụ phân tích chuỗi dữ liệu nhận được để trích xuất chỉ số nhiệt độ và độ ẩm, từ đó vi điều khiển sẽ gửi dữ liệu theo một trong hai format đã cấu hình.

4.2.4. Lưu vào cơ sở dữ liệu

```
void save_to_database(int temperature, int humidity) {
    FILE* fp;
    time_t now;
    struct tm* timeinfo;
    char timestamp[64];
```

```
    time(&now);
    timeinfo = localtime(&now);
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", timeinfo);
    // kiểm tra sự tồn tại của file
    int file_exists = 0;
    fp = fopen(DB_FILE, "r");
    if (fp != NULL) {
        file_exists = 1;
        fclose(fp);
    }

    fp = fopen(DB_FILE, "a");
    if (fp == NULL) {
        printf("[ERROR] Cannot open database file\n");
        return;
    }
    // ghi header (nếu tạo file mới)
    if (!file_exists) {
        fprintf(fp, "Timestamp,Temperature(C),Humidity(%%)\n");
    }
    // ghi data
    fprintf(fp, "%s,%d,%d\n", timestamp, temperature, humidity);
    fclose(fp);

    printf("[DB] Data saved: %s | T:%d C | H:%d %%\n",
           timestamp, temperature, humidity);
}
```

Trước khi ghi dữ liệu hệ thống sẽ kiểm tra file tồn tại, nếu không có sẵn hệ thống sẽ tạo file mới với header đã cài đặt mặc định trong cấu hình để ghi dữ liệu. Hàm `save_to_database()` sẽ ghi dữ liệu vào file CSV (gồm timestamp – thời gian, temperature – nhiệt độ, humidity – độ ẩm) lấy thời gian hệ thống và format thành chuỗi theo dạng "YYYY-MM-DD HH:MM:SS" lần lượt là "năm-tháng-ngày giờ-phút-giây". File mới sẽ được mở ở chế độ "a" (append) để luôn cập nhật dữ liệu ở cuối mảng mà không xóa hay đè lên dữ liệu cũ.

4.3. CƠ SỞ DỮ LIỆU SQLite

4.3.1. Cấu trúc và khởi tạo

Bảng 4.1. Cấu trúc cơ sở dữ liệu SQLite

id	int	PRIMARY KEY AUTOINCREMENT	Khóa chính tự động tăng
timestamp	text	NOT NULL	Thời gian đo (YYYY-MM-DD HH:MM:SS)

temperature	int	NOT NULL	Nhiệt độ (°C)
humidity	int	NOT NULL	Độ ẩm (%)

Khởi tạo hàm để mở hoặc tạo file database ghi dữ liệu, hàm `sqline3_exec()` được sử dụng để thực hiện câu lệnh tạo bảng nếu bảng chưa tồn tại.

```
int init_sqlite_database(void) {
    int rc = sqlite3_open("sensor_data.db", &db);
    if (rc != SQLITE_OK) {
        printf("[ERROR] Cannot open database\n");
        return -1;
    }

    const char* sql = "CREATE TABLE IF NOT EXISTS sensor_data ("
                      "id INTEGER PRIMARY KEY AUTOINCREMENT, "
                      "timestamp TEXT NOT NULL, "
                      "temperature INTEGER NOT NULL, "
                      "humidity INTEGER NOT NULL);";

    char* err_msg;
    rc = sqlite3_exec(db, sql, 0, 0, &err_msg);
    if (rc != SQLITE_OK) {
        printf("[ERROR] SQL error: %s\n", err_msg);
        sqlite3_free(err_msg);
        return -1;
    }

    printf("[OK] SQLite database ready\n");
    return 0;
}
```

4.3.2. Lưu trữ dữ liệu vào SQLite

```
void save_to_sqlite(int temperature, int humidity) {
    time_t now;
    char timestamp[64];
    time(&now);
    strftime(timestamp, 64, "%Y-%m-%d %H:%M:%S", localtime(&now));

    sqlite3_stmt* stmt;
    const char* sql = "INSERT INTO sensor_data "
                      "(timestamp, temperature, humidity) "
                      "VALUES (?, ?, ?)";

    sqlite3_prepare_v2(db, sql, -1, &stmt, 0);
    sqlite3_bind_text(stmt, 1, timestamp, -1, SQLITE_STATIC);
    sqlite3_bind_int(stmt, 2, temperature);
    sqlite3_bind_int(stmt, 3, humidity);

    if (sqlite3_step(stmt) == SQLITE_DONE) {
```

```
        printf("[SQLite] Data saved: %s | T:%d | H:%d\n",
               timestamp, temperature, humidity);
    }

    sqlite3_finalize(stmt);
}
```

CHƯƠNG 5: QUẢNG BÁ BLE (BLE ADVERTISING)

5.1. GIỚI THIỆU

Quảng bá BLE (BLE Advertising) là chế độ chia sẻ dữ liệu broadcast nhưng chỉ theo hướng một chiều của BLE (Bluetooth Low Energy). Điều này cho phép thiết bị phát tín hiệu quảng bá chứa dữ liệu cần truyền thông qua bluetooth mà không cần kết nối trực tiếp.

EFR32xG21 sẽ đóng vai trò là một thiết bị phát (advertiser) quảng bá dữ liệu, trong phạm vi đồ án này là chỉ số nhiệt độ và độ ẩm của môi trường, đến thiết bị nhận (scanner) như điện thoại, tablet,... có cài đặt ứng dụng Si Connect. Ưu điểm của phương thức truyền dữ liệu này là có thể tiết kiệm năng lượng và có thể có nhiều thiết bị quét đồng thời để nhận cùng một thông tin, nhưng gói dữ liệu gửi đi sẽ bị giới hạn 31 bytes/packet.

Bảng 5.1. Cấu trúc cơ bản của một gói tin BLE Advertising

Preamble	**Access Address**	**PDU Header**	**Advertising Address**	**Advertising Data**			**CRC**
				Flags	Manufacture Data	Device Name	
1 byte	4 bytes	2 bytes	6 bytes	0 – 31 bytes			3 bytes
Đồng bộ tín hiệu	Địa chỉ truy cập quảng bá	Độ dài gói tin	Địa chỉ MAC	Dữ liệu			Kiểm tra lỗi

5.2. CẤU TRÚC CUSTOM ADVERTISING DATA

Cấu trúc dữ liệu quảng bá được định nghĩa trong file `custom_adv.h` có thể tùy chỉnh theo nhu cầu sử dụng.

```
/**
 * @brief BLE Advertising packet structure for AHT20 sensor
 *
 * Total size calculation:
 * - Flags: 3 bytes (len + type + value)
 * - Manufacturer Data: 1 + len_manuf bytes
 *   + Type (1 byte) = 0xFF
 *   + Company ID (2 bytes)
 *   + Temperature (2 bytes)
 *   + Humidity (2 bytes)
 * - Device Name: 1 + 1 + name_length bytes
 */
```

Cấu trúc gồm 3 phần chính:

- *Flags (3 bytes)*

```
  // Flags (3 bytes)
  uint8_t len_flags;       // Length = 0x02
  uint8_t type_flags;      // Type = 0x01 (Flags)
```

```
  uint8_t val_flags;       // Value = 0x06
```

Gồm 1 byte lưu giá trị độ dài, 1 byte để định nghĩa kiểu dữ liệu là flag và 1 byte lưu giá trị của cờ.

- *Manufacture data (6 bytes)*

```
  // Manufacturer Specific Data
  uint8_t len_manuf;       // Length = 0x07 (1 + 2 + 2 + 2)
  uint8_t type_manuf;      // Type = 0xFF (Manufacturer Specific Data)
  uint8_t company_LO;      // Company ID Low Byte
  uint8_t company_HI;      // Company ID High Byte

  // Sensor Data (4 bytes total)
  uint8_t temp_hi;         // Temperature High Byte (MSB)
  uint8_t temp_lo;         // Temperature Low Byte (LSB)
  uint8_t humi_hi;         // Humidity High Byte (MSB)
  uint8_t humi_lo;         // Humidity Low Byte (LSB)
```

Chỉ số nhiệt độ và độ ẩm sẽ được lưu trữ và mã hóa theo dạng BCD (tương đương sử dụng 2 bytes cho mỗi giá trị)

- *Device name (variable)*

```
  // Device Name
  uint8_t len_name;        // Length = 1 + string length
  uint8_t type_name;       // Type = 0x08 (Shortened) or 0x09 (Complete)
  char name[NAME_MAX_LENGTH]; // Device name string

  // Bookkeeping
  char dummy;              // Padding
  uint8_t data_size;       // Total advertising data size
```

Dữ liệu quảng bá sẽ được đóng gói thành một gói tin có cấu trúc như vừa trình bày, dung lượng bộ nhớ cho từng phần có thể tùy chỉnh phụ thuộc vào nhu cầu sử dụng.

5.3. CẬP NHẬT CẢM BIẾN

```
void update_adv_data_sensor(CustomAdv_t *pData,
                            uint8_t advertising_set_handle,
                            float temperature,
                            float humidity)
{
  sl_status_t sc;

  // Làm tròn thành số nguyên
  int temp_int = (int)roundf(temperature);
  int humi_int = (int)roundf(humidity);

  // CHUYỂN SANG BCD DECIMAL FORMAT
  // Temperature: 30 -> 0x0030 (BCD)
  if (temp_int >= 100) {
    // Nếu > 99°C, dùng 2 byte BCD: temp_hi = tens, temp_lo = ones
```

```
    pData->temp_hi = int_to_bcd(temp_int / 100);
    pData->temp_lo = int_to_bcd(temp_int % 100);
  } else {
    // Nếu < 100°C, chỉ dùng 1 byte BCD
    pData->temp_hi = 0x00;
    pData->temp_lo = int_to_bcd(temp_int);
  }

  // Humidity: 65 -> 0x0065 (BCD)
  if (humi_int >= 100) {
    pData->humi_hi = int_to_bcd(humi_int / 100);
    pData->humi_lo = int_to_bcd(humi_int % 100);
  } else {
    pData->humi_hi = 0x00;
    pData->humi_lo = int_to_bcd(humi_int);
  }

  // Update BLE advertising data
  sc = sl_bt_legacy_advertiser_set_data(advertising_set_handle,
                                        0,
                                        pData->data_size,
                                        (const uint8_t *)pData);
  app_assert(sc == SL_STATUS_OK,
             "[E: 0x%04x] Failed to update advertising data\n",
             (int)sc);
}
```

5.4. XỬ LÝ SỰ KIỆN BLE (BLE EVENT HANDLER)

Hệ thống xử lý các sự kiện BLE thông qua hàm `sl_bt_on_event()` với cấu trúc switch-case. Khi khởi động, event `sl_bt_evt_system_boot_id` được kích hoạt và thực hiện chuỗi khởi tạo: lấy địa chỉ MAC 6 bytes bằng `sl_bt_system_get_identity_address()` và in ra UART theo format hex, tạo advertising set bằng `sl_bt_advertiser_create_set()` để quản lý cấu hình, cấu hình timing với interval 1600 units (1 giây) thông qua `sl_bt_advertiser_set_timing()`, enable cả ba kênh (37, 38, 39) bằng channel map = 7, khởi tạo packet với `fill_adv_packet()`, và cuối cùng cập nhật dữ liệu sensor nếu đã sẵn sàng.

Khi event `sl_bt_evt_connection_closed_id` xảy ra, hệ thống cập nhật lại dữ liệu sensor vì giá trị có thể đã thay đổi trong lúc kết nối, sau đó restart advertising bằng `sl_bt_legacy_advertiser_start()` với mode connectable_scannable vì advertising tự động dừng khi có kết nối.

```
void sl_bt_on_event(sl_bt_msg_t *evt) {
    switch (SL_BT_MSG_ID(evt->header)) {
        case sl_bt_evt_system_boot_id:
            sl_bt_system_get_identity_address(&address, &address_type);
            sl_bt_advertiser_create_set(&advertising_set_handle);
```

```
            sl_bt_advertiser_set_timing(advertising_set_handle,      1600,
1600, 0, 0);
            sl_bt_advertiser_set_channel_map(advertising_set_handle, 7);
            fill_adv_packet(&sData, 0x06, 0x02D5, "NHOM18");
            if (sensor_initialized) {
                update_adv_data_sensor(&sData, advertising_set_handle,
                                       g_temperature, g_humidity);
            }
            break;

        case sl_bt_evt_connection_closed_id:
            if (sensor_initialized) {
                update_adv_data_sensor(&sData, advertising_set_handle,
                                       g_temperature, g_humidity);
            }
            sl_bt_legacy_advertiser_start(advertising_set_handle,

sl_bt_advertiser_connectable_scannable);
            break;
    }
}
```

5.5. HÀM START ADVERTISING

Hàm `start_adv()` thực hiện hai nhiệm vụ chính. Đầu tiên là load 19 bytes dữ liệu vào BLE stack thông qua `sl_bt_legacy_advertiser_set_data()` với tham số packet type = 0 (advertising data), BLE stack sẽ copy dữ liệu vào bộ nhớ nội bộ nên buffer gốc có thể thay đổi sau đó. Thứ hai là khởi động broadcast bằng `sl_bt_legacy_advertiser_start()` với mode connectable_scannable cho phép vừa broadcast vừa chấp nhận kết nối, sau đó BLE stack sẽ broadcast packet mỗi 1 giây trên cả ba kênh cho đến khi có kết nối hoặc bị dừng thủ công.

```
void start_adv(CustomAdv_t *pData, uint8_t advertising_set_handle) {
    sl_status_t sc;

    // Load dữ liệu vào BLE stack
    sc = sl_bt_legacy_advertiser_set_data(advertising_set_handle, 0,
                                          pData->data_size,
                                          (const uint8_t *)pData);
    if (sc != SL_STATUS_OK) {
        uart_print("[E: 0x%04x] Failed to set data\n", (int)sc);
    }

    // Bắt đầu broadcast
    sc = sl_bt_legacy_advertiser_start(advertising_set_handle,

sl_bt_advertiser_connectable_scannable);
    if (sc != SL_STATUS_OK) {
```

```
        uart_print("[E: 0x%04x] Failed to start\n", (int)sc);
    }
}
```

CHƯƠNG 6: CHƯƠNG TRÌNH CHÍNH

6.1. TỔNG QUAN

Chương trình chính tổng hợp tất cả module con (cảm biến AHT20, giao tiếp UART, LCD, BLE) thành một ứng dụng điều khiển hoàn chỉnh. Kiến trúc gồm ba thành phần chính: khối khởi tạo chứa tất cả cấu hình ban đầu (bao gồm cả thiết bị ngoại vi), khối xử lý chịu trách nhiệm quản lý sự kiện ngắt từ UART và Timer, khối vòng lặp hỗ trợ cập nhật trạng thái hệ thống theo chu kì thiết lập.

6.2. KIẾN TRÚC PHẦN MỀM

6.2.1. UART

Khởi tạo

Hàm `init_uart()` cấu hình giao tiếp UART giữa vi điều khiển và PC ở tốc độ 115200 baud.

```
void init_uart(void)
{
  USART_InitAsync_TypeDef init = USART_INITASYNC_DEFAULT;

  CMU_ClockEnable(cmuClock_GPIO, true);
  CMU_ClockEnable(cmuClock_USART0, true);

  GPIO_PinModeSet(UART_TX_PORT, UART_TX_PIN, gpioModePushPull, 1);
  GPIO_PinModeSet(UART_RX_PORT, UART_RX_PIN, gpioModeInput, 0);
  GPIO_PinModeSet(VCOM_ENABLE_PORT, VCOM_ENABLE_PIN, gpioModePushPull, 1);

  init.enable = usartEnable;
  init.refFreq = 0;
  init.baudrate = 115200;
  init.oversampling = usartOVS16;
  init.databits = usartDatabits8;
  init.parity = usartNoParity;
  init.stopbits = usartStopbits1;

// Routing UART đến GPIO
  GPIO->USARTROUTE[0].TXROUTE = (UART_TX_PORT <<
_GPIO_USART_TXROUTE_PORT_SHIFT)
                              | (UART_TX_PIN <<
_GPIO_USART_TXROUTE_PIN_SHIFT);
  GPIO->USARTROUTE[0].RXROUTE = (UART_RX_PORT <<
_GPIO_USART_RXROUTE_PORT_SHIFT)
                              | (UART_RX_PIN <<
_GPIO_USART_RXROUTE_PIN_SHIFT);
  GPIO->USARTROUTE[0].ROUTEEN = GPIO_USART_ROUTEEN_RXPEN |
GPIO_USART_ROUTEEN_TXPEN;
```

```
  USART_InitAsync(USART0, &init);

  // Clear any pending interrupts
  USART_IntClear(USART0, _USART_IF_MASK);

  // Enable RX interrupt
  USART_IntEnable(USART0, USART_IF_RXDATAV);
  NVIC_ClearPendingIRQ(USART0_RX_IRQn);
  NVIC_EnableIRQ(USART0_RX_IRQn);
}
```

`USART_IntEnable()` sử dụng `RXDATAV` (Input Data Valid) để kích hoạt ngắt mỗi khi receive buffer nhận được một byte dữ liệu mới và `NVIC` (Nested Vectored Interrupt Controller) được cấu hình với mức ưu tiên mặc định cho phép ngắt `USART0_RX` xảy ra.

- *UART Interrupt Handler*

```
void USART0_RX_IRQHandler(void)
{
  // Kiểm tra có phải RX interrupt không
  uint32_t flags = USART_IntGet(USART0);

  if (flags & USART_IF_RXDATAV) {
    uart_rx_data = USART_Rx(USART0); // đọc từ buffer
    uart_data_ready = true;
    uart_rx_count++;  // Debug counter

    // Clear interrupt flag
    USART_IntClear(USART0, USART_IF_RXDATAV);
  }
}
```

Nhiệm vụ chính của hàm `USART0_RX_IRQHandler()` là đọc dữ liệu từ receive buffer và đặt cờ, hàm được gọi tự động mỗi khi có dữ liệu mới từ UART.

- *Xử lý lệnh UART*

Hàm `process_uart_command()` chứa các lệnh điều khiển sẽ nhận tín hiệu từ PC và thực hiện hành động tương ứng. Hàm được gọi từ vòng lặp chính trong hàm `main()` khi biến `uart_data_ready = true`.

6.2.2. Timer Callback

Hàm `sensor_timer_callback()` được gọi định kỳ bởi `app_timer` để thực hiện việc đọc cảm biến và cập nhật BLE.

```
static void sensor_timer_callback(app_timer_t *timer, void *data) {
    (void)timer;
    (void)data;
```

```
    float raw_temp, raw_humi;

    // Đọc dữ liệu từ AHT20
    sl_status_t sc = aht20_read_data(sl_i2cspm_DCD,
                                     &raw_temp, &raw_humi);  //float

    if (sc == SL_STATUS_OK) {
        // Làm tròn thành số nguyên
        g_temperature = (int)roundf(raw_temp);
        g_humidity = (int)roundf(raw_humi);

        // In qua UART
        uart_printf("[INFO] Temp: %d C, Humidity: %d %%\r\n",
                   g_temperature, g_humidity);

        // Cập nhật BLE advertising data
        if (advertising_set_handle != 0xff) {
            update_adv_data_sensor(&adv,
                                   advertising_set_handle,
                                   (float)g_temperature,
                                   (float)g_humidity);
        }
    } else {
        uart_printf("[ERROR] Failed to read sensor\r\n");
        sensor_initialized = false;
    }
}
```

6.2.3. Vòng lặp chính

Hàm `app_process_action()` là vòng lặp chính được gọi liên tục của chương trình để thực hiện các lệnh điều khiển cũng như cập nhật trạng thái đo lường. Hàm sẽ xử lý hai tác vụ chính là lệnh điều khiển từ PC qua giao thức UART và cập nhật hiển thị chỉ số lên màn hình LCD.

```
void app_process_action(void) {
// Xử lý UART command (nếu có)
process_uart_command();
// Cập nhật LCD (nếu dữ liệu thay đổi)
memlcd_app_process_action();
}
```

Hàm `process_uart_command()` chỉ làm việc khi biến `uart_data_ready = true`, hàm `memlcd_app_process_action()` chỉ cập nhật dữ liệu trên màn hình LCD khi dữ liệu có sự thay đổi (so với dữ liệu cũ).

6.3. CHẾ ĐỘ HOẠT ĐỘNG

Auto-logging mode: Timer đọc dữ liệu từ cảm biến theo chu kỳ theo biến `g_sensor_period`. Dữ liệu sẽ được gửi qua UART theo format đã cấu hình, sau đó PC sẽ tự động phân tích dữ liệu và lưu vào database. Chế độ này cho phép ghi nhận dữ liệu liên tục mà không cần can thiệp.

Command mode: Lệnh điều khiển từ PC có thể được gửi bất cứ lúc nào. Các lệnh sẽ được xử lý ngay lập tức và không làm gián đoạn chế độ Auto-logging.

CHƯƠNG 7: CHẠY THỬ NGHIỆM

7.1. BẢNG ĐIỀU KHIỂN VÀ GIAO DIỆN CONSOLE

```
========================================
   NHOM18 - AHT20 Monitoring System
   CSV + SQLite Data Storage
========================================

[OK] SQLite database opened: sensor_data.db
[OK] SQLite table 'sensor_data' ready
[OK] Port opened: \\.\COM4 at 115200 baud
[INFO] Auto-listening mode ACTIVE
[INFO] Data will be saved to CSV and SQLite
[INFO] Press any key to send command (or 'q' to quit)


========================================
      NHOM18 - AHT20 Control Panel
========================================
1 - Read sensor data NOW
2 - Increase sensor period (+1s)
3 - Decrease sensor period (-1s)
4 - Increase BLE interval (+100ms)
5 - Decrease BLE interval (-100ms)
6 - Show current configuration
7 - Show database statistics
h - Show this menu
q - Quit program
========================================
Mode: AUTO-LISTENING + COMMAND MODE
Data Storage: CSV + SQLite
```

Hình 7.1. Bảng điều khiển

```
========================================
[INFO] Temp: 32 C, Humidity: 59 %
[CSV] Data saved: 2025-12-06 15:04:40 | T:32 C | H:59 %
[SQLite] Data saved: 2025-12-06 15:04:40 | T:32 C | H:59 %
[INFO] Temp: 33 C, Humidity: 65 %
[CSV] Data saved: 2025-12-06 15:04:45 | T:33 C | H:65 %
[SQLite] Data saved: 2025-12-06 15:04:45 | T:33 C | H:65 %
[INFO] Temp: 34 C, Humidity: 81 %
[CSV] Data saved: 2025-12-06 15:04:50 | T:34 C | H:81 %
[SQLite] Data saved: 2025-12-06 15:04:50 | T:34 C | H:81 %
[INFO] Temp: 35 C, Humidity: 85 %
[CSV] Data saved: 2025-12-06 15:04:55 | T:35 C | H:85 %
[SQLite] Data saved: 2025-12-06 15:04:55 | T:35 C | H:85 %
```

Hình 7.2. Kết quả hiển thị trên giao diện Console

7.2. CHẠY CÁC LỆNH COMMAND

Khi chạy command ‘1’, dữ liệu từ cảm biến sẽ được đọc ngay lập tức.

```
[CMD] Sending command: '1'
[RESPONSE] Waiting for device response...
[INFO] Temp: 34 C, Humidity: 86 %
```

Hình 7.3. Kết quả khi chạy Command ‘1’

Khi chạy command ‘2’, thời gian đọc dữ liệu trên cảm biến sẽ tăng lên 1s.

```
[CMD] Sending command: '2'
[RESPONSE] Waiting for device response...

[DEBUG] RX: '2' (0x32) | Count: 2/2
Sensor period set to: 6 seconds
[DEBUG] Stopping timer...
[DEBUG] Starting timer with new period...
[OK] Timer restarted: 6 seconds

[INFO] Temp: 33 C, Humidity: 66 %
[CSV] Data saved: 2025-12-06 15:05:20 | T:33 C | H:66 %
[SQLite] Data saved: 2025-12-06 15:05:20 | T:33 C | H:66 %
[INFO] Temp: 33 C, Humidity: 64 %
[CSV] Data saved: 2025-12-06 15:05:26 | T:33 C | H:64 %
[SQLite] Data saved: 2025-12-06 15:05:26 | T:33 C | H:64 %
```

Hình 7.4. Kết quả khi chạy Command ‘2’

Khi chạy command '3', thời gian đọc dữ liệu trên cảm biến sẽ giảm xuống 1s.

```
[DEBUG] RX: '3' (0x33) | Count: 4/4
Sensor period set to: 4 seconds
[DEBUG] Restarting timer...
[OK] Timer restarted: 4 seconds

[INFO] Temp: 33 C, Humidity: 61 %
[CSV] Data saved: 2025-12-06 15:05:39 | T:33 C | H:61 %
[SQLite] Data saved: 2025-12-06 15:05:39 | T:33 C | H:61 %
[INFO] Temp: 33 C, Humidity: 61 %
[CSV] Data saved: 2025-12-06 15:05:43 | T:33 C | H:61 %
[SQLite] Data saved: 2025-12-06 15:05:43 | T:33 C | H:61 %
[INFO] Temp: 33 C, Humidity: 61 %
[CSV] Data saved: 2025-12-06 15:05:47 | T:33 C | H:61 %
[SQLite] Data saved: 2025-12-06 15:05:47 | T:33 C | H:61 %
[INFO] Temp: 33 C, Humidity: 60 %
[CSV] Data saved: 2025-12-06 15:05:51 | T:33 C | H:60 %
[SQLite] Data saved: 2025-12-06 15:05:51 | T:33 C | H:60 %
```

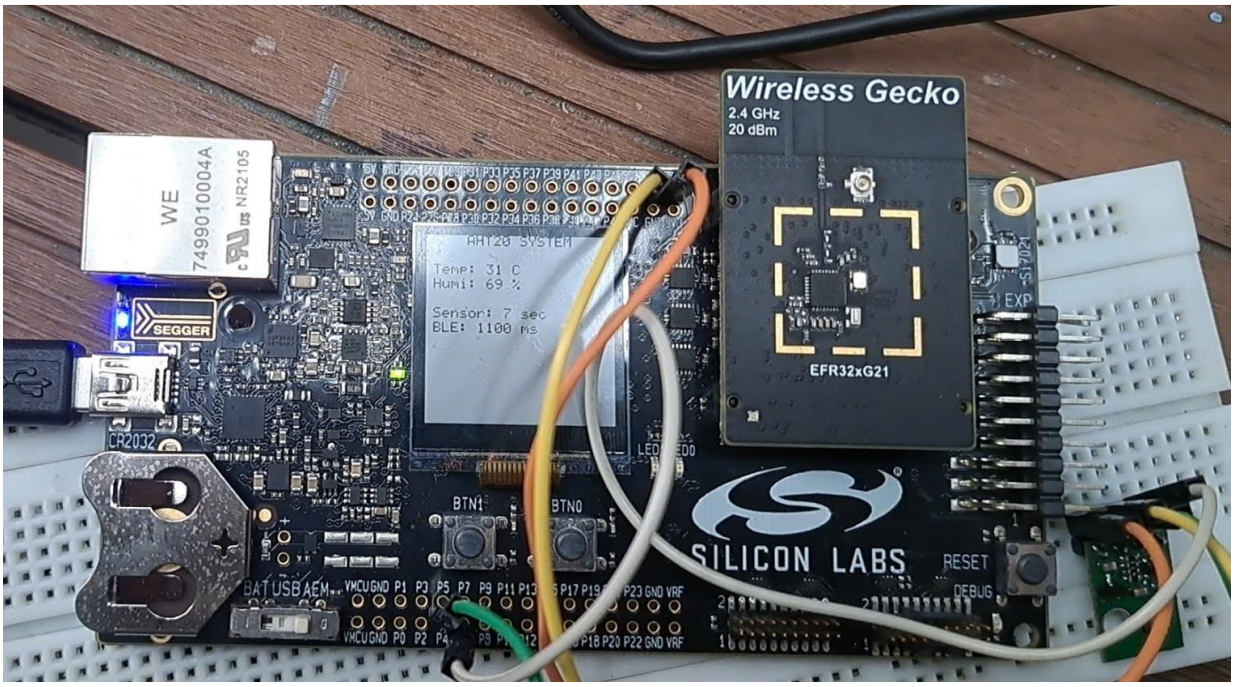
Hình 7.5. Kết quả khi chạy Command '3'

Khi chạy command '4', thời gian quảng bá dữ liệu qua BLE sẽ tăng lên 100ms. Tương tự khi chạy commanad '5', thời gian quảng bá dữ liệu qua BLE sẽ giảm xuống 100ms.

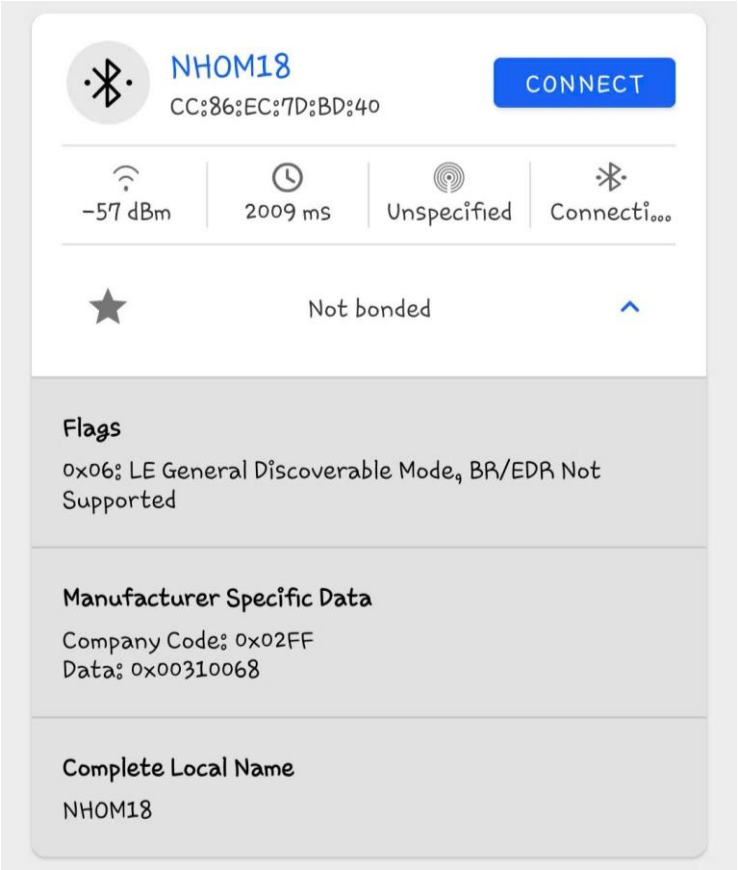
```
[DEBUG] RX: '4' (0x34) | Count: 6/6
[DEBUG] Increasing BLE interval...
[OK] BLE interval: 1100 ms

[INFO] Temp: 33 C, Humidity: 60 %
[CSV] Data saved: 2025-12-06 15:06:00 | T:33 C | H:60 %
[SQLite] Data saved: 2025-12-06 15:06:00 | T:33 C | H:60 %

[CMD] Sending command: '5'
[RESPONSE] Waiting for device response...

[DEBUG] RX: '5' (0x35) | Count: 7/7
[DEBUG] Decreasing BLE interval...
[OK] BLE interval: 1000 ms
```

Hình 7.6. Kết quả khi chạy Command '4' và '5'

Hình 7.7. Giảm thời gian quảng bá BLE 100ms



Hình 7.8. Tăng thời gian quảng bá BLE 100ms



Hình 7.9. Kết quả quảng bá trên BLE

Khi chạy command ‘6’, màn hình Console sẽ hiển thị cấu hình hiện tại.

```
[CMD] Sending command: '6'
[RESPONSE] Waiting for device response...

[DEBUG] RX: '6' (0x36) | Count: 8/8

=== Current Configuration ===
Sensor Period: 5 seconds
BLE Interval: 1000 ms
Temperature: 33 C
Humidity: 60 %
Sensor Status: OK
UART RX Count: 8 (Processed: 8)
============================
```

Hình 7.10. Kết quả khi chạy Command ‘6’

Khi chạy command '7', hệ thống sẽ cho ra bảng thống kê từ cơ sở dữ liệu file `sensor_data.db`.

```
========================================
      STATISTICS FROM DATABASE
========================================
Total Records: 23
----------------------------------------
Temperature:
  Average: 33.2 C
  Min: 32 C
  Max: 35 C
----------------------------------------
Humidity:
  Average: 66.6 %
  Min: 59 %
  Max: 85 %
========================================
```

Hình 7.11. Kết quả khi chạy Command '7'

Khi chạy command 'q', chương trình sẽ kết thúc và dữ liệu sẽ được in ra database dưới dạng bảng Excel.

```
[INFO] Exiting...
[OK] SQLite database closed
[OK] Port closed
[OK] All data saved to:
     - CSV: sensor_data.csv
     - SQLite: sensor_data.db
PS D:\NAM4_HK1\ThucHanhGiaoTiepMayTinh\AHT20_new>
```

Hình 7.12. Kết quả chạy Command 'q'

7.3. KẾT QUẢ

	A	B	C	D	E	F	G	H	I	J
1	Timestamp	Temperature(C)	Humidity(%)							
2	12/3/2025 9:46	30	73							
3	12/3/2025 9:46	30	73							
4	12/3/2025 9:46	31	76							
5	12/3/2025 9:46	32	78							
6	12/3/2025 9:46	32	77							
7	12/3/2025 9:46	33	76							
8	12/3/2025 9:46	33	74							
9	12/3/2025 9:46	32	70							
10	12/3/2025 9:46	32	69							
11	12/3/2025 9:46	32	69							
12	12/3/2025 9:47	31	69							
13	12/3/2025 9:47	31	69							
14	12/3/2025 9:47	31	70							
15	12/3/2025 9:47	31	70							
16	12/3/2025 9:47	31	70							
17	12/3/2025 9:47	31	70							
18	12/3/2025 9:47	31	70							
19	12/3/2025 9:47	30	71							
20	12/3/2025 9:47	30	71							
21	12/3/2025 9:47	30	71							
22	12/3/2025 9:47	30	71							
23	12/3/2025 9:47	30	71							
24	12/3/2025 9:48	30	71							

Hình 7.13. Dữ liệu được in ra trong file CSV

Structure | Data | Constraints | Indexes | Triggers | DDL

sensor_data Table name: sensor_data ☐ WITHOUT ROWID ☐ STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER	🔑							NULL
2	timestamp	TEXT					🔑			NULL
3	temperature	INTEGER					🔑			NULL
4	humidity	INTEGER					🔑			NULL

Hình 7.14. Cấu trúc file `sensor_data.db`

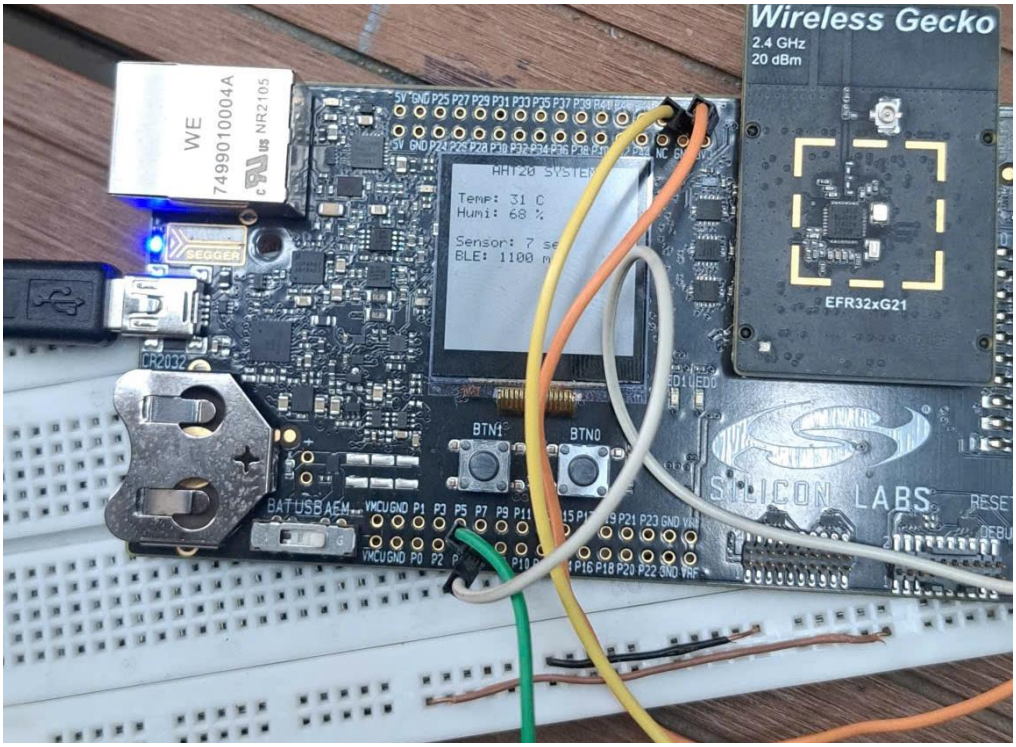
Structure | Data | Constraints | Indexes | Triggers | DDL

Grid view | Form view

Filter data — Total rows loaded: 25

	id	timestamp	temperature	humidity
1	1	2025-12-06 14:52:13	33	59
2	2	2025-12-06 14:52:18	33	61
3	3	2025-12-06 14:52:23	34	79
4	4	2025-12-06 14:52:28	34	84
5	5	2025-12-06 14:52:33	33	78
6	6	2025-12-06 15:04:40	32	59
7	7	2025-12-06 15:04:45	33	65
8	8	2025-12-06 15:04:50	34	81
9	9	2025-12-06 15:04:55	35	85
10	10	2025-12-06 15:05:05	33	76
11	11	2025-12-06 15:05:10	33	71
12	12	2025-12-06 15:05:20	33	66
13	13	2025-12-06 15:05:26	33	64
14	14	2025-12-06 15:05:39	33	61
15	15	2025-12-06 15:05:43	33	61
16	16	2025-12-06 15:05:47	33	61
17	17	2025-12-06 15:05:51	33	60
18	18	2025-12-06 15:06:00	33	60
19	19	2025-12-06 15:06:05	33	60
20	20	2025-12-06 15:06:10	33	60
21	21	2025-12-06 15:06:15	33	60
22	22	2025-12-06 15:06:20	33	60
23	23	2025-12-06 15:06:25	33	60

Hình 7.15. Ghi dữ liệu trên SQLite



Hình 7.16. Kết quả hiển thị lên LCD

CHƯƠNG 8: KẾT LUẬN

8.1. TỔNG KẾT

Nhìn chung, nhóm đã hoàn thành tương đối đầy đủ các yêu cầu của đồ án. Hệ thống đo lường nhiệt độ và độ ẩm được xây dựng có tính tiện lợi, dễ sử dụng và thể hiện được tiềm năng phát triển trong các ứng dụng thực tế. Sau quá trình kiểm thử và hiệu chỉnh, hệ thống hoạt động ổn định, dữ liệu thu được có độ sai lệch nhỏ và đáp ứng được các chức năng cốt lõi như hiển thị lên LCD, truyền dữ liệu qua UART đến PC và quảng bá thông tin qua BLE.

8.2. HẠN CHẾ

Xuyên suốt quá trình thực hiện đồ án, hệ thống đo lường nhiệt độ và độ ẩm có những mặt hạn chế nhất định.

Trong thời gian triển khai, PC không đọc được chỉ số dữ liệu nhiệt độ - độ ẩm từ cảm biến dẫn đến không thể hiển thị kết quả lên màn hình LCD và không tìm thấy thiết bị khi quảng bá đến ứng dụng Simplicity Connect trên thiết bị di động. Nhưng sau khi khắc phục lỗi sai kể trên, hệ thống hoạt động tương đối ổn định và dữ liệu có sai lệch không đáng kể.

Do giới hạn về thời gian và kinh phí, chỉ tích hợp một cảm biến không quá phức tạp theo dõi và tính toán hai chỉ của môi trường.

8.3. HƯỚNG PHÁT TRIỂN

Trong tương lai, chúng em mong muốn trước tiên có thể cải thiện hiệu suất hoạt động của hệ thống đồng thời tích hợp thêm nhiều tính năng cá nhân hóa trải nghiệm sử dụng với mục đích tiếp cận gần hơn với nhu cầu thị trường.

Chúng em sẽ nghiên cứu sâu thêm về những bộ lọc thích hợp nhằm loại bỏ tối đa tín hiệu nhiễu giúp hệ thống hoạt động ổn định hơn, giảm sai số và cho hiệu suất cao hơn.

Thêm vào đó, nhóm em dự tính sẽ kết hợp thêm linh kiện phần cứng hỗ trợ theo dõi dữ liệu từ xa trên trang web và gửi cảnh báo đến người dùng khi chỉ số môi trường vượt ngưỡng cho phép.